Monday 26th of July 2004, 04:30:56 pm. Posted in: **Embedded**. 8804 words.

Processor specific code:

uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/arch.c

Machine dependent fixups are added in this file. arch_hard_reset may contain code to reset the processor, perhaps by using the watchdog timer.

A machine_desc structure with the architecture features for the machine must also be specified in this file. This is done by the MACHINE_START and MACHINE_END macros. The first argument to MACHINE_START is the machine type (MACH_TYPE_xxx) defined in the file mach-types, while the second argument is the machine name. The maintainer macro holds the port maintainer's contact information and is used purely for documentary purposes in the source code.

INITIRQ defines the IRQ initialization function, genarch_init_irq, in the file uclinux/linux-2.4.x/arch/armnommu/kernel/irq-arch.c. This function in turn expands a macro irq_init_irq declared in uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/irq.h. The BOOT_PARAMS macro specifies the location in DRAM where the boot parameters from the bootloader may be found. The parameter structure is created by bootloaders such as U-boot and armboot to pass the console parameters, initial ramdisk etc to the kernel.

A full list of features is declared in uclinux/linux-2.4.x/include/asm-armnommu/mach/arch.h.

```
/*
* linux/arch/armnommu/mach-s3c44b0x/arch.c
*
* Architecture specific fixups. This is where any
* parameters in the params struct are fixed up, or
* any additional architecture specific information
* is pulled from the params struct.
*/
#include <linux/tty.h>
```

```
#include <linux/delay.h>
#include <linux/pm.h>
#include <linux/init.h>
#include <asm/elf.h>
#include <asm/setup.h>
#include <asm/mach-types.h>
#include <asm/mach/arch.h>

extern void genarch_init_irq(void);

MACHINE_START(ACME, "ACME SOHO-B0X")
        MAINTAINER("Nemo Physche <nemo@dotgeek.org>")
        INITIRQ(genarch_init_irq)
        BOOT_PARAMS(0x0c000100)
MACHINE_END

void arch_hard_reset(void)
{
        /*
         * TODO: activate the hardware reset
         */
        while (1)
                ;
}
```

uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/irq.c

This file implements functions to initialize interrupts and mask/unmask/acknowledge irqs for the s3c44b0x processor. These functions are called by the kernel when required. The necessary glue is in the irq_init_irq function in uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/irq.h

```
/*
* linux/arch/armnommu/mach-s3c44b0x/irq.c
*
```

```
* Interrupt handling
*
*/
#include <linux/init.h>


#include <asm/mach/irq.h>
#include <asm/hardware.h>
#include <asm/io.h>
#include <asm/irq.h>
#include <asm/system.h>


void s3c44b0x_mask_irq(unsigned int irq)
{
        INT_DISABLE(irq);
}


void s3c44b0x_unmask_irq(unsigned int irq)
{
        INT_ENABLE(irq);
}


void s3c44b0x_mask_ack_irq(unsigned int irq)
{
        INT_DISABLE(irq);
}


void s3c44b0x_int_init()
{
        IntPend = 0x03FFFFFF;
        IntMode = INT_MODE_IRQ;
        INT_ENABLE(INT_GLOBAL);
}
```

uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/time.c

Setup of the timer is done in the inline function setup_timer in the file uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/time.h. In time.c, the timer interrupt handler and the function gettimeoffset is implemented.

The timer interrupt handler for the s3c44b0x processor requires no special fixups, so control is passed directly to the kernel's timer interrupt handler. During setup of the timer in the function setup_timer, the kernel is intimated of the existence of this function.

CLOCKS_PER_USEC is the processor clock speed / 10^6. The processor clock speed CONFIG_ARM_CLK is defined in uclinux/linux-2.4.x/.config which is created by uclinux/linux-2.4.x/arch/armnommu/config.in. Gettimeoffset returns the number of microseconds since the last timer interrupt occurred.

```
/*
* linux/arch/armnommu/mach-s3c44b0x/time.c
*
* Timer interrupt handler
*
*/
#include <linux/time.h>
#include <linux/timex.h>
#include <linux/types.h>
#include <linux/sched.h>
#include <asm/io.h>
#include <asm/arch/hardware.h>

#define CLOCKS_PER_USEC        (CONFIG_ARM_CLK/1000000)

#if ((CLOCKS_PER_USEC*1000000) != CONFIG_ARM_CLK)
#warning "clock rate is not in whole MHz, gettimeoffset will give wrong results"
#endif

unsigned long acme_gettimeoffset (void)
{
```

```
        return (CSR_READ(rTCNTB5) / CLOCKS_PER_USEC);
}


void acme_timer_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
        do_timer(regs);
}
```

uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/dma.c

This is a placeholder since DMA support for this processor has not been added.

```
/*
* linux/arch/armnommu/mach-s3c44b0x/dma.c
*
* This program is free software; you can redistribute it and/or modify
* it under the terms of the GNU General Public License as published by
* the Free Software Foundation; either version 2 of the License, or
* (at your option) any later version.
*
* This program is distributed in the hope that it will be useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
* GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public License
* along with this program; if not, write to the Free Software
* Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
*/
#include <asm/page.h>
#include <asm/pgtable.h>
#include <asm/dma.h>
#include <asm/mach/dma.h>
```

```
void arch_dma_init(dma_t *dma)

{

}
```

uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/Makefile

A simple Makefile to compile the C files in the directory to create acme.o.

```
#

# Makefile for the linux kernel.

#

# Note! Dependencies are done automagically by 'make dep', which also

# removes any old dependencies. DON'T put your own dependencies here

# unless it's something special (ie not a .c file).

USE_STANDARD_AS_RULE := true


O_TARGET                := acme.o


# Object file lists.


obj-y                           := $(patsubst %.c, %.o, $(wildcard *.c))

obj-m                           :=

obj-n                           :=

obj-                            :=


export-objs                     :=


include $(TOPDIR)/Rules.make
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/hardware.h

Header file declaring hardware registers on the s3c44b0x processor.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/hardware.h
*
*/
#ifndef __ASM_ARCH_HARDWARE_H
#define __ASM_ARCH_HARDWARE_H

/* Force to little endian mode*/
#ifdef __BIG_ENDIAN
#undef __BIG_ENDIAN
#endif
/*
* define S3C44b0b CPU master clock
*/
#define MHz                1000000
#define fMCLK_MHz          (CONFIG_ARM_CLK)
#define fMCLK              (fMCLK_MHz / MHz)

/*
* ASIC Address Definition
*/

#define Base_Addr        0x1c00000

#define VPint            *(volatile unsigned int *)
#define VPshort          *(volatile unsigned short *)
#define VPchar           *(volatile unsigned char *)

#ifndef CSR_WRITE
# define CSR_WRITE(addr,data)        (VPint(addr) = (data))
#endif
```

```
#ifndef CSR_READ
# define CSR_READ(addr)        (VPint(addr))
#endif

#ifndef CAM_Reg
# define CAM_Reg(x)            (VPint(CAMBASE+(x*0x4)))
#endif

/* System */
#define SYSCFG                   ( 0x1c00000)

/* Cache */
#define rNCACHBE0                ( 0x1c00004)
#define rNCACHBE1                ( 0x1c00008)

/* Bus control */
#define rSBUSCON                 ( 0x1c40000)

/* Memory control */
#define rBWSCON                  ( 0x1c80000)
#define rBANKCON0                ( 0x1c80004)
#define rBANKCON1                ( 0x1c80008)
#define rBANKCON2                ( 0x1c8000c)
#define rBANKCON3                ( 0x1c80010)
#define rBANKCON4                ( 0x1c80014)
#define rBANKCON5                ( 0x1c80018)
#define rBANKCON6                ( 0x1c8001c)
#define rBANKCON7                ( 0x1c80020)
#define rREFRESH                 ( 0x1c80024)
#define rBANKSIZE                ( 0x1c80028)
#define rMRSRB6                  ( 0x1c8002c)
#define rMRSRB7                  ( 0x1c80030)
```

```
/* UART */
#define rULCON0                  ( 0x1d00000)
#define rULCON1                  ( 0x1d04000)
#define rUCON0                   ( 0x1d00004)
#define rUCON1                   ( 0x1d04004)
#define rUFCON0                  ( 0x1d00008)
#define rUFCON1                  ( 0x1d04008)
#define rUMCON0                  ( 0x1d0000c)
#define rUMCON1                  ( 0x1d0400c)
#define rUTRSTAT0                ( 0x1d00010)
#define rUTRSTAT1                ( 0x1d04010)
#define rUERSTAT0                ( 0x1d00014)
#define rUERSTAT1                ( 0x1d04014)
#define rUFSTAT0                 ( 0x1d00018)
#define rUFSTAT1                 ( 0x1d04018)
#define rUMSTAT0                 ( 0x1d0001c)
#define rUMSTAT1                 ( 0x1d0401c)
#define rUBRDIV0                 ( 0x1d00028)
#define rUBRDIV1                 ( 0x1d04028)

#ifdef __BIG_ENDIAN
#define rUTXH0                   ( 0x1d00023)
#define rUTXH1                   ( 0x1d04023)
#define rURXH0                   ( 0x1d00027)
#define rURXH1                   ( 0x1d04027)
#define WrUTXH0(ch)        ( *(volatile unsigned char *)0x1d00023)=(unsigned char)(ch)
#define WrUTXH1(ch)        ( *(volatile unsigned char *)0x1d04023)=(unsigned char)(ch)
#define RdURXH0()        (*(volatile unsigned char *)(0x1d00027))
#define RdURXH1()        (*(volatile unsigned char *)(0x1d04027))
#define UTXH0                (0x1d00020+3) /*byte_access address by BDMA*/
#define UTXH1                   (0x1d04020+3)
#define URXH0                   (0x1d00024+3)
#define URXH1                   (0x1d04024+3)
```

```
#else /*Little Endian*/
#define rUTXH0                  ( 0x1d00020)
#define rUTXH1                  ( 0x1d04020)
#define rURXH0                  ( 0x1d00024)
#define rURXH1                  ( 0x1d04024)
#define WrUTXH0(ch)        (*(volatile unsigned char *)0x1d00020)=(unsigned char)(ch)
#define WrUTXH1(ch)        (*(volatile unsigned char *)0x1d04020)=(unsigned char)(ch)
#define RdURXH0()        (*(volatile unsigned char *)0x1d00024)
#define RdURXH1()        (*(volatile unsigned char *)0x1d04024)
#define UTXH0               (0x1d00020) /*byte_access address by BDMA*/
#define UTXH1                 (0x1d04020)
#define URXH0                 (0x1d00024)
#define URXH1                 (0x1d04024)
#endif

/* SIO */
#define rSIOCON               ( 0x1d14000)
#define rSIODAT               ( 0x1d14004)
#define rSBRDR                ( 0x1d14008)
#define rIVTCNT               ( 0x1d1400c)
#define rDCNTZ                ( 0x1d14010)

/* IIS */
#define rIISCON               ( 0x1d18000)
#define rIISMOD               ( 0x1d18004)
#define rIISPSR               ( 0x1d18008)
#define rIISFCON              ( 0x1d1800c)

#ifdef __BIG_ENDIAN
#define rIISFIF               ((volatile unsigned short *)0x1d18012)

#else /*Little Endian*/
#define rIISFIF               ((volatile unsigned short *)0x1d18010)
#endif
```

```
/* I/O PORT */
#define rPCONA                  ( 0x1d20000)
#define rPDATA                  ( 0x1d20004)

#define rPCONB                  ( 0x1d20008)
#define rPDATB                  ( 0x1d2000c)

#define rPCONC                  ( 0x1d20010)
#define rPDATC                  ( 0x1d20014)
#define rPUPC                   ( 0x1d20018)

#define rPCOND                  ( 0x1d2001c)
#define rPDATD                  ( 0x1d20020)
#define rPUPD                   ( 0x1d20024)

#define rPCONE                  ( 0x1d20028)
#define rPDATE                  ( 0x1d2002c)
#define rPUPE                   ( 0x1d20030)

#define rPCONF                  ( 0x1d20034)
#define rPDATF                  ( 0x1d20038)
#define rPUPF                   ( 0x1d2003c)

#define rPCONG                  ( 0x1d20040)
#define rPDATG                  ( 0x1d20044)
#define rPUPG                   ( 0x1d20048)

#define rSPUCR                  ( 0x1d2004c)
#define rEXTINT                 ( 0x1d20050)
#define rEXTINPND               ( 0x1d20054)

/* WATCHDOG */
#define rWTCON                  ( 0x1d30000)
```

```
#define rWTDAT                  ( 0x1d30004)
#define rWTCNT                  ( 0x1d30008)

/* ADC */
#define rADCCON                 ( 0x1d40000)
#define rADCPSR                 ( 0x1d40004)
#define rADCDAT                 ( 0x1d40008)

/* Timer */
#define rTCFG0                  ( 0x1d50000)
#define rTCFG1                  ( 0x1d50004)
#define rTCON                   ( 0x1d50008)

#define rTCNTB0                 ( 0x1d5000c)
#define rTCMPB0                 ( 0x1d50010)
#define rTCNTO0                 ( 0x1d50014)

#define rTCNTB1                 ( 0x1d50018)
#define rTCMPB1                 ( 0x1d5001c)
#define rTCNTO1                 ( 0x1d50020)

#define rTCNTB2                 ( 0x1d50024)
#define rTCMPB2                 ( 0x1d50028)
#define rTCNTO2                 ( 0x1d5002c)

#define rTCNTB3                 ( 0x1d50030)
#define rTCMPB3                 ( 0x1d50034)
#define rTCNTO3                 ( 0x1d50038)

#define rTCNTB4                 ( 0x1d5003c)
#define rTCMPB4                 ( 0x1d50040)
#define rTCNTO4                 ( 0x1d50044)
```

```
#define rTCNTB5                 ( 0x1d50048)
#define rTCNTO5                 ( 0x1d5004c)

/* IIC */
#define rIICCON                 ( 0x1d60000)
#define rIICSTAT                ( 0x1d60004)
#define rIICADD                 ( 0x1d60008)
#define rIICDS                  ( 0x1d6000c)

/* RTC */
#ifdef __BIG_ENDIAN
#define rRTCCON                 ( 0x1d70043)
#define rRTCALM                 ( 0x1d70053)
#define rALMSEC                 ( 0x1d70057)
#define rALMMIN                 ( 0x1d7005b)
#define rALMHOUR                ( 0x1d7005f)
#define rALMDAY                 ( 0x1d70063)
#define rALMMON                 ( 0x1d70067)
#define rALMYEAR                ( 0x1d7006b)
#define rRTCRST                 ( 0x1d7006f)
#define rBCDSEC                 ( 0x1d70073)
#define rBCDMIN                 ( 0x1d70077)
#define rBCDHOUR                ( 0x1d7007b)
#define rBCDDAY                 ( 0x1d7007f)
#define rBCDDATE                ( 0x1d70083)
#define rBCDMON                 ( 0x1d70087)
#define rBCDYEAR                ( 0x1d7008b)
#define rTICINT                 ( 0x1d7008e)
#else
#define rRTCCON                 ( 0x1d70040)
#define rRTCALM                 ( 0x1d70050)
#define rALMSEC                 ( 0x1d70054)
#define rALMMIN                 ( 0x1d70058)
#define rALMHOUR                ( 0x1d7005c)
```

```
#define rALMDAY                ( 0x1d70060)
#define rALMMON                ( 0x1d70064)
#define rALMYEAR               ( 0x1d70068)
#define rRTCRST                ( 0x1d7006c)
#define rBCDSEC                ( 0x1d70070)
#define rBCDMIN                ( 0x1d70074)
#define rBCDHOUR               ( 0x1d70078)
#define rBCDDAY                ( 0x1d7007c)
#define rBCDDATE               ( 0x1d70080)
#define rBCDMON                ( 0x1d70084)
#define rBCDYEAR               ( 0x1d70088)
#define rTICINT                ( 0x1d7008c)
#endif

/* Clock & Power management */
#define rPLLCON                ( 0x1d80000)
#define rCLKCON                ( 0x1d80004)
#define rCLKSLOW               ( 0x1d80008)
#define rLOCKTIME              ( 0x1d8000c)

/* INTERRUPT */
#define rINTCON                ( 0x1e00000)
#define rINTPND                ( 0x1e00004)
#define rINTMOD                ( 0x1e00008)
#define rINTMSK                ( 0x1e0000c)

#define rI_PSLV                ( 0x1e00010)
#define rI_PMST                ( 0x1e00014)
#define rI_CSLV                ( 0x1e00018)
#define rI_CMST                ( 0x1e0001c)
#define rI_ISPR                ( 0x1e00020)
#define rI_ISPC                ( 0x1e00024)
```

```
#define rF_ISPR                ( 0x1e00038)
#define rF_ISPC                ( 0x1e0003c)

/* LCD */
#define rLCDCON1               ( 0x1f00000)
#define rLCDCON2               ( 0x1f00004)
#define rLCDCON3               ( 0x1f00040)
#define rLCDSADDR1             ( 0x1f00008)
#define rLCDSADDR2             ( 0x1f0000c)
#define rLCDSADDR3             ( 0x1f00010)
#define rREDLUT                ( 0x1f00014)
#define rGREENLUT              ( 0x1f00018)
#define rBLUELUT               ( 0x1f0001c)
#define rDP1_2                 ( 0x1f00020)
#define rDP4_7                 ( 0x1f00024)
#define rDP3_5                 ( 0x1f00028)
#define rDP2_3                 ( 0x1f0002c)
#define rDP5_7                 ( 0x1f00030)
#define rDP3_4                 ( 0x1f00034)
#define rDP4_5                 ( 0x1f00038)
#define rDP6_7                 ( 0x1f0003c)
#define rDITHMODE              ( 0x1f00044)

/* ZDMA0 */
#define rZDCON0                ( 0x1e80000)
#define rZDISRC0               ( 0x1e80004)
#define rZDIDES0               ( 0x1e80008)
#define rZDICNT0               ( 0x1e8000c)
#define rZDCSRC0               ( 0x1e80010)
#define rZDCDES0               ( 0x1e80014)
#define rZDCCNT0               ( 0x1e80018)

/* ZDMA1 */
#define rZDCON1                ( 0x1e80020)
```

```
#define rZDISRC1                ( 0x1e80024)
#define rZDIDES1                ( 0x1e80028)
#define rZDICNT1                ( 0x1e8002c)
#define rZDCSRC1                ( 0x1e80030)
#define rZDCDES1                ( 0x1e80034)
#define rZDCCNT1                ( 0x1e80038)

/* BDMA0 */
#define rBDCON0                 ( 0x1f80000)
#define rBDISRC0                ( 0x1f80004)
#define rBDIDES0                ( 0x1f80008)
#define rBDICNT0                ( 0x1f8000c)
#define rBDCSRC0                ( 0x1f80010)
#define rBDCDES0                ( 0x1f80014)
#define rBDCCNT0                ( 0x1f80018)

/* BDMA1 */
#define rBDCON1                 ( 0x1f80020)
#define rBDISRC1                ( 0x1f80024)
#define rBDIDES1                ( 0x1f80028)
#define rBDICNT1                ( 0x1f8002c)
#define rBDCSRC1                ( 0x1f80030)
#define rBDCDES1                ( 0x1f80034)
#define rBDCCNT1                ( 0x1f80038)

#define UART_BASE0              rULCON0
#define UART_BASE1              rULCON1

#if DEBUG_CONSOLE == 0
#define WRITEUART(ch) WrUTXH0(ch)
        #define DEBUG_UARTLCON_BASE        rULCON0
        #define DEBUG_UARTCONT_BASE        rUCON0
        #define DEBUG_UARTBRD_BASE        rUBRDIV0
        #define DEBUG_CHK_STAT_BASE        rUTRSTAT0
```

```
#elif DEBUG_CONSOLE == 1
/*
#define WRITEUART(ch) WrUTXH1(ch)
        #define DEBUG_TX_BUFF_BASE        UTXBUF1
        #define DEBUG_RX_BUFF_BASE        URXBUF1
        #define DEBUG_UARTLCON_BASE        rULCON1
        #define DEBUG_UARTCONT_BASE        rUCON1
        #define DEBUG_UARTBRD_BASE        rUBRDIV1
        #define DEBUG_CHK_STAT_BASE        rUTRSTAT1*/
#endif
#define DEBUG_TX_DONE_CHECK_BIT    (0X02)

#define INT_MODE_IRQ           0x00000000
#define INT_MODE_FIQ           0x03FFFFFF
#define INT_MASK_DIS           0x03FFFFFF
#define INT_MASK_ENA           0x00000000

#define IntMode                (VPint(rINTMOD))
#define IntPend                (VPint(rI_ISPC))
#define IntMask                (VPint(rINTMSK))

#define INT_ENABLE(n)          IntMask &= ~(1<<(n))
#define INT_DISABLE(n)         IntMask |= (1<<(n))
#define CLEAR_PEND_INT(n)      IntPend = (1<<(n))

#define HARD_RESET_NOW()       { arch_hard_reset(); }

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/irqs.h

Declaration of all interrupt requests present on the s3c44b0x processor. NR_IRQS must be declared to be the total number of irqs available.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/irqs.h
*
*/
#ifndef __ASM_ARCH_IRQS_H__
#define __ASM_ARCH_IRQS_H__

#define NR_IRQS                         (26)

#define INT_ADC                         (0)
#define INT_RTC                         (1)
#define INT_UTXD1                       (2)
#define INT_UTXD0                       (3)
#define INT_SIO                         (4)
#define INT_IIC                         (5)
#define INT_URXD1                       (6)
#define INT_URXD0                       (7)
#define INT_TIMER5                      (8)
#define INT_TIMER4                      (9)
#define INT_TIMER3                      (10)
#define INT_TIMER2                      (11)
#define INT_TIMER1                      (12)
#define INT_TIMER0                      (13)
#define INT_UERR01                      (14)
#define INT_WDT                         (15)
#define INT_BDMA1                       (16)
#define INT_BDMA0                       (17)
#define INT_ZDMA1                       (18)
#define INT_ZDMA0                       (19)
#define INT_TICK                        (20)
#define INT_EINT4567                    (21)
#define INT_EINT3                       (22)
#define INT_EINT2                       (23)
#define INT_EINT1                       (24)
```

```
#define INT_EINT0                       (25)
#define INT_GLOBAL                      (26)

#define IRQ_TIMER                       INT_TIMER5

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/irq.h

As mentioned before the macro irq_init_irq expands in the function genarch_init_irq, in the file uclinux/linux-2.4.x/arch/armnommu/kernel/irq-arch.c. The macro initializes the interrupt controller of the s3c44b0x processor and associates the mask/unmask/ack functions (defined in uclinux/linux-2.4.x/arch/armnommu/mach-s3c44b0x/irq.c) with the irq_desc structure used by the linux kernel.

When an IRQ occurs, control is passed to the function vector_IRQ in the file uclinux/linux-2.4.x/arch/armnommu/kernel/entry-armv.S. __irq_svc is then called, which gets the interrupt request number (using the macro get_irqnr_and_base) before passing it as an argument to the C function do_IRQ in uclinux/linux-2.4.x/arch/armnommu/kernel/irq.c.

The bit corresponding to the IRQ being currently serviced is set in the s3c44b0x processor register I_ISPR. In the current port, the macro get_irqnr_and_base gets the value of the register I_ISPR instead of the actual interrupt. Hence in do_IRQ, the code in fixup_irq returns the actual interrupt number that has been raised, after clearing the interrupt pending bit.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/irq.h
*
*/
#ifndef __ASM_ARCH_IRQ_H__
#define __ASM_ARCH_IRQ_H__

#include <asm/hardware.h>
#include <asm/io.h>
```

```
#include <asm/mach/irq.h>
#include <asm/arch/irqs.h>

static __inline__ int fixup_irq(int irq)
{
        int i = 0;

        for (i = 0; i < 26; i++)
        {
                if (irq == 1)
                        break;
                irq = irq >> 1;
        }

        CLEAR_PEND_INT(i);
        return i;
}

extern void s3c44b0x_mask_irq(unsigned int irq);
extern void s3c44b0x_unmask_irq(unsigned int irq);
extern void s3c44b0x_mask_ack_irq(unsigned int irq);
extern void s3c44b0x_int_init(void);

static __inline__ void irq_init_irq(void)
{
        unsigned long flags;
        int irq;

        save_flags_cli(flags);
        s3c44b0x_int_init();
        restore_flags(flags);

        for (irq = 0; irq < NR_IRQS; irq++) {
                irq_desc[irq].valid = 1;
```

```
                irq_desc[irq].probe_ok = 1;
                irq_desc[irq].mask_ack = s3c44b0x_mask_ack_irq;
                irq_desc[irq].mask = s3c44b0x_mask_irq;
                irq_desc[irq].unmask = s3c44b0x_unmask_irq;
        }
}
#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/time.h

The function setup_timer configures the hardware timer to generate an interrupt every 10 milliseconds. This value depends on HZ, declared in uclinux/linux-2.4.x/include/asm-armnommu/param.h. Timer 5 is configured in auto-reload mode and the timer interrupt is enabled. The function s3c44b0x_timer_interrupt is configured as the interrupt handler.

Timer 5 is configured with a timer counter value of 1250, a prescaler value of 59, and a divider value of 8 which causes an interrupt every 10 milliseconds on the 60Mhz processor (i.e: (1 / (60000000 / 8)) * (59 + 1) * 1250 = 0.001 seconds or 10 milliseconds).

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/time.h
*
*/
#ifndef __ASM_ARCH_TIME_H
#define __ASM_ARCH_TIME_H

#include <asm/uaccess.h>
#include <asm/io.h>
#include <asm/hardware.h>
#include <asm/arch/timex.h>

extern struct irqaction timer_irq;
```

```
extern unsigned long s3c44b0x_gettimeoffset(void);
extern void s3c44b0x_timer_interrupt(int irq, void *dev_id, struct pt_regs *regs);

void __inline__ setup_timer (void)
{
        /* enable IRQ, and using non-vectored interrupt mode */
        CSR_WRITE(rINTCON,5);

        /* set all IRQ mode */
        CSR_WRITE(rINTMOD,0x0);

        /* set precaler2 for timer5, clear all other precaler
         * config 0
         */
        CSR_WRITE(rTCFG0,(59 << 16));

        /* set interrupt mode and MUX 1/8 for timer5
         * config 1
         */
        CSR_WRITE(rTCFG1,(0x02 << 20));

        /* fill The timer count, every 10ms have a interrupt */
        CSR_WRITE(rTCNTB5,1250);

        /* manual update, and set autoload mode
        * set Interrupt Mode and MUX 1/8 for timer5
        */
        CSR_WRITE(rTCON,((1 << 26)+(1 << 25)));

        /* set autoload mode and start timer */
        CSR_WRITE(rTCON,((1 << 26)+(1 << 24)));

        CLEAR_PEND_INT(IRQ_TIMER);
        INT_ENABLE(IRQ_TIMER);
```

```
        gettimeoffset = s3c44b0x_gettimeoffset;
        timer_irq.handler = s3c44b0x_timer_interrupt;
        setup_arm_irq(IRQ_TIMER, &timer_irq);
}

#endif
```

If it is required to modify the value of HZ in uclinux/linux-2.4.x/include/asm-armnommu/param.h, the timer counter value must also be modified accordingly (i.e: there must be HZ timer interrupts every second, that is an interrupt every 1 / HZ seconds). Additionally, the function udelay in uclinux/linux-2.4.x/arch/armnommu/lib/delay.S must be modified to reflect this change. A description of this modification is at (http://lists.arm.linux.org.uk/pipermail/linux-arm/2001-April/001171.html). To summarize, modify the right shift value 11, in the instruction (mov r1, r1, lsr #11). For example to 9 for HZ = 400.

```
LC0:                .word        SYMBOL_NAME(loops_per_jiffy)
/*
* 0 <= r0 <= 2000
*/
ENTRY(udelay)
                mov        r2, #0x6800
                orr        r2, r2, #0x00db
                mul        r1, r0, r2
                ldr        r2, LC0
                ldr        r2, [r2]
                mov        r1, r1, lsr #11
                mov        r2, r2, lsr #11
                mul        r0, r1, r2
                movs       r0, r0, lsr #6
                RETINSTR(moveq,pc,lr)
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/memory.h

TASK_SIZE is the maximum size in bytes of a user process on the 32-bit processor. It's main use is on processors with a virtual memory management system. In the nommu case, TASK_SIZE is set to the last accessible memory location in DRAM. PHYS_OFFSET is the physical start address of the DRAM, which also equals PAGE_OFFSET since without virtual memory, the base address is the first usable page in memory.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/memory.h
*
*/
#ifndef __ASM_ARCH_MEMORY_H
#define __ASM_ARCH_MEMORY_H

#define TASK_SIZE        (0x01a00000UL + 0x0c000000UL)
#define TASK_SIZE_26     TASK_SIZE

#define PHYS_OFFSET      (DRAM_BASE)
#define PAGE_OFFSET      PHYS_OFFSET
#define END_MEM          (DRAM_BASE + DRAM_SIZE)

#define __virt_to_phys__is_a_macro
#define __virt_to_phys(vpage) ((unsigned long) (vpage))
#define __phys_to_virt__is_a_macro
#define __phys_to_virt(ppage) ((void *) (ppage))

#define __virt_to_bus__is_a_macro
#define __virt_to_bus(vpage) ((unsigned long) (vpage))
#define __bus_to_virt__is_a_macro
#define __bus_to_virt(ppage) ((void *) (ppage))

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/processor.h

TASK_SIZE should be the same value as defined in uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/memory.h.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/processor.h
*
*/
#ifndef __ASM_ARCH_PROCESSOR_H
#define __ASM_ARCH_PROCESSOR_H

#define TASK_SIZE (0x01a00000UL + 0x0c000000UL)

#define EISA_bus 0
#define EISA_bus__is_a_macro
#define MCA_bus 0
#define MCA_bus__is_a_macro

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/io.h

PCIO_BASE and IO_SPACE_LIMIT refer to the first and last addresses of the IO map respectively. The memset*, memcpy*, read* and write* macros are taken from uclinux/linux-2.4.x/include/asm-armnommu/io.h, and is needed for the Memory Technology Device (MTD) subsystem to provide flash support.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/io.h
*
*/
#ifndef __ASM_ARM_ARCH_IO_H
#define __ASM_ARM_ARCH_IO_H
```

```
/*
* kernel/resource.c uses this to initialize the global ioport_resource struct
* which is used in all calls to request_resource(), allocate_resource(), etc.
* --gmcnutt
*/
#define IO_SPACE_LIMIT 0xffffffff

/*
* If we define __io then asm/io.h will take care of most of the inb & friends
* macros. It still leaves us some 16bit macros to deal with ourselves, though.
* We don't have PCI or ISA on the dsc21 so I dropped __mem_pci & __mem_isa.
* --gmcnutt
*/
#define PCIO_BASE 0
#define __io(a) (PCIO_BASE + (a))
#define __arch_getw(a) (*(volatile unsigned short *)(a))
#define __arch_putw(v,a) (*(volatile unsigned short *)(a) = (v))

/*
* Defining these two gives us ioremap for free. See asm/io.h.
* --gmcnutt
*/
#define iomem_valid_addr(iomem,sz) (1)
#define iomem_to_phys(iomem) (iomem)

/*
* These functions are needed for mtd/maps/physmap.c
* --rp
*/
#ifndef memset_io
#define memset_io(a,b,c) _memset_io((a),(b),(c))
#endif
#ifndef memcpy_fromio
#define memcpy_fromio(a,b,c) _memcpy_fromio((a),(b),(c))
```

```
#endif

#ifndef memcpy_toio

#define memcpy_toio(a,b,c) _memcpy_toio((a),(b),(c))

#endif


#ifndef __mem_pci

/* Implement memory read/write functions (needed for mtd) */

#define readb(addr) __arch_getb(addr)

#define readw(addr) __arch_getw(addr)

#define readl(addr) __arch_getl(addr)

#define writeb(v,addr) __arch_putb(v,addr)

#define writew(v,addr) __arch_putw(v,addr)

#define writel(v,addr) __arch_putl(v,addr)


#define eth_io_copy_and_sum(a,b,c,d) __readwrite_bug("eth_io_copy_and_sum")


#define check_signature(io,sig,len) (0)


#endif


#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/vmalloc.h

A place holder file, not used.

```
/*

* linux/include/asm-armnommu/arch-s3c44b0x/vmalloc.h

*

*/

#ifndef __ARCH_ASM_VMALLOC_H__

#define __ARCH_ASM_VMALLOC_H__
```

```
/*
* Just any arbitrary offset to the start of the vmalloc VM area: the
* current 8MB value just means that there will be a 8MB "hole" after the
* physical memory until the kernel virtual memory starts. That means that
* any out-of-bounds memory accesses will hopefully be caught.
* The vmalloc() routines leaves a hole of 4kB between each vmalloced
* area for the same reason. ; )
*/
#define VMALLOC_OFFSET    (8*1024*1024)
#define VMALLOC_START     (((unsigned long)high_memory + VMALLOC_OFFSET) & ~(VMALLOC_OFFSET-
1))
#define VMALLOC_VMADDR(x) ((unsigned long)(x))
#define VMALLOC_END       (PAGE_OFFSET + 0x10000000)


#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/system.h

arch_idle is the code executed by the idle process when there is no other process to run. This function should put the hardware in a low power mode. arch_reset is used to reset the processor during a system reset.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/system.h
*
*/
#ifndef __ASM_ARCH_SYSTEM_H
#define __ASM_ARCH_SYSTEM_H


static inline void arch_idle(void)
{
        while (!current->need_resched && !hlt_counter)
                cpu_do_idle(IDLE_WAIT_SLOW);
}
```

```
extern inline void arch_reset(char mode)
{
        switch (mode) {
                case 's':
                        /* software reset jump to address 0x0*/
                        cpu_reset(0);
                        break;
                case 'h':
                        /* hardware reset watchdog timer? */
                        break;
        }
}

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/uncompress.c

The functions in this file are used as a debugging aid during the initial stages of the kernel bootup before the serial driver has been setup. s3c44b0x_decomp_setup sets up the first serial port at 115200 baud, and s3c44b0x_puts is used to output a string through this port.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/uncompress.c
*
*/
#include <asm/hardware.h>

static int s3c44b0x_decomp_setup()
{
        int i;

        /* setup io pins */
        CSR_WRITE(rPCONE, (2 << 2 | 2 << 4));
```

```
        /* fifo disable */
        CSR_WRITE(rUFCON0, 0x0);
        CSR_WRITE(rUMCON0, 0x0);

        CSR_WRITE(rULCON0, 0x00000003 | 0x00000000 |
                  0x00000000 | 0x00000000);

        /* 115200 BAUD */
        CSR_WRITE(rUBRDIV0, 32);

        /* Start channel
         * Enable the UART in UCON and Unmask the RX interrupt!
         */
        CSR_WRITE(rUCON0, 0x005);

        for (i = 1000; i > 0; i--);
}

static int s3c44b0x_putc(char c)
{
        WRITEUART(c);
        while(!(CSR_READ(DEBUG_CHK_STAT_BASE) &
                DEBUG_TX_DONE_CHECK_BIT));

        if(c == '\n')
                s3c44b0x_putc('\r');
}

static void s3c44b0x_puts(const char *s)
{
        while(*s != '\0')
                s3c44b0x_putc(*s++);
}
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/uncompress.h

The debugging macros puts and arch_decomp_setup are mapped to the architecture specific code. puts may be used to output debug messages at the initial stages of the kernel bootup.

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/uncompress.h
*
*/
#ifndef __ASM_ARCH_UNCOMPRESS_H
#define __ASM_ARCH_UNCOMPRESS_H

#include <asm/arch/uncompress.c>

#define puts(s)                     s3c44b0x_puts(s)
#define arch_decomp_wdog()
#define arch_decomp_setup()         s3c44b0x_decomp_setup()

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/shmparam.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/shmparam.h
*
*/
#ifndef __ASM_ARCH_SHMPARAM_H
#define __ASM_ARCH_SHMPARAM_H

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/param.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/param.h
*
*/
#ifndef __ARCH_ASM_PARAM_H__
#define __ARCH_ASM_PARAM_H__

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/keyboard.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/keyboard.h
*
* (Dummy) Keyboard driver definitions for ARM
*
*/
#ifndef __ASM_ARM_ARCH_KEYBOARD_H
#define __ASM_ARM_ARCH_KEYBOARD_H
/*
* Required by drivers/char/keyboard.c.
* --gmcnutt
*/
#define kbd_setkeycode(sc,kc) (-EINVAL)
#define kbd_getkeycode(sc) (-EINVAL)
#define kbd_translate(sc,kcp,rm) ({ *(kcp) = (sc); 1; })
#define kbd_unexpected_up(kc) (0200)
#define kbd_leds(leds)
#define kbd_init_hw()
#define kbd_enable_irq()
#define kbd_disable_irq()

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/dma.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/dma.h
*
*/
#ifndef __ASM_ARCH_DMA_H
#define __ASM_ARCH_DMA_H

#define MAX_DMA_ADDRESS                0x03000000
#define MAX_DMA_CHANNELS               13

#endif /* _ASM_ARCH_DMA_H */
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/a.out.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/a.out.h
*
*/
#ifndef __ASM_ARCH_A_OUT_H
#define __ASM_ARCH_A_OUT_H

#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/arch-s3c44b0x/timex.h

```
/*
* linux/include/asm-armnommu/arch-s3c44b0x/timex.h
*
*/
#ifndef __ASM_ARCH_TIMEX_H
#define __ASM_ARCH_TIMEX_H
```

```
#endif
```

uclinux/linux-2.4.x/include/asm-armnommu/proc-armv/system.h

The s3c44b0x processor uses vector interrupts and therefore requires an Interrupt Vector Table. When an interrupt occurs, the processor jumps to the corresponding location in the IVT. For the s3c44b0x this is at location 0x0, which addresses a flash device. Hence we maintain two IVTs, one for the processor and the other calling generic interrupt handlers in linux.

The processor IVT at location 0x0 is part of the bootloader and is stored in flash. Entries in the processor IVT jump to the linux IVT located at the start of DRAM at address 0x0c000000.

```
_start:         b          start_code
                ldr        pc, _undefined_instruction
                ldr        pc, _software_interrupt
                ldr        pc, _prefetch_abort
                ldr        pc, _data_abort
                ldr        pc, _not_used
                ldr        pc, _irq
                ldr        pc, _fiq
_undefined_instruction:     .word   0x0c000004
_software_interrupt:        .word   0x0c000008
_prefetch_abort:            .word   0x0c00000C
_data_abort:                .word   0x0c000010
_not_used:                  .word   0x0c000014
_irq:                       .word   0x0c000018
_fiq:                       .word   0x0c00001C


start_code:
```

The linux IVT is then mapped to the start of DRAM by setting vectors_base to 0x0c000000 in this file. The mapping is done in the function trap_init in uclinux/linux-2.4.x/arch/armnommu/kernel/traps.c

```
..

..

..

#ifdef __ARM_ARCH_4__

#ifdef CONFIG_ARCH_ACME

#define vectors_base() (0x0c000000)

#else

#define vectors_base() ((cr_alignment & CR_V) ? 0xffff0000 : 0)

#endif

#else

#define vectors_base() (0)

#endif

..

..

..
```

Jump to page: 1 2 3 4 **5 6**

Comments (1) | **Permalink**